

**ORBENYX**

Software &amp; Product Engineering

CONFIDENTIAL — CLIENT COPY

TECHNICAL PROJECT DOCUMENTATION

# Kee — Platform Architecture & Implementation Reference

A complete engineering reference for the Kee platform: system architecture, project structure, technology stack, full API surface, database schema, authentication and authorization, third-party integrations, deployment topology, and security posture — for engineers taking over, auditing, or extending the system.

**PREPARED BY**

Orbenyx Pvt. Ltd.  
12 Residency Road, Coimbatore, Tamil  
Nadu 641018, India

**CONTACT**

hello@orbenyx.studio  
+91 00000 00000  
www.orbenyx.studio

**DOCUMENT**

Rev. 1.0 — Draft  
Issued 18 August 2026 · Placeholder  
fields for client review

# TABLE OF CONTENTS

*Ordered from system-level architecture down to individual endpoints and workflows.*

## ARCHITECTURE OVERVIEW

- 1.1 System overview & request flow
- 1.2 Technology stack
- 1.3 Project structure

## DATA MODEL

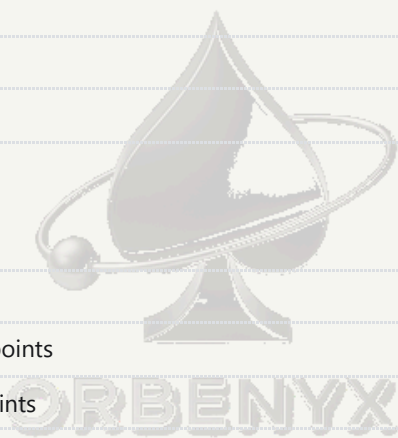
- 2.1 Database schema overview
- 2.2 Core models & relationships

## AUTHENTICATION & AUTHORIZATION

- 3.1 Login & token issuance
- 3.2 Guards, roles & rate limiting
- 3.3 OTP delivery & Firebase phone auth

## API REFERENCE

- 4.1 Authentication endpoints
- 4.2 Shop & shop category endpoints
- 4.3 Customer & compliance record endpoints
- 4.4 Machines, promotions & keys endpoints
- 4.5 Advertisements & notifications
- 4.6 Payments & platform configuration
- 4.7 Public (unauthenticated) endpoints



## INTEGRATIONS & CONFIGURATION

- 5.1 Third-party integrations
- 5.2 Environment variables

## DEPLOYMENT & OPERATIONS

- 6.1 Deployment topology
- 6.2 Build & release pipeline

## SECURITY CONSIDERATIONS

- 7.1 Implemented controls
- 7.2 Known limitations & hardening notes

## TECHNICAL WORKFLOWS

- 8.1 Shop registration & payment sequence
- 8.2 Request authentication flow
- 8.3 Customer visit & compliance record flow

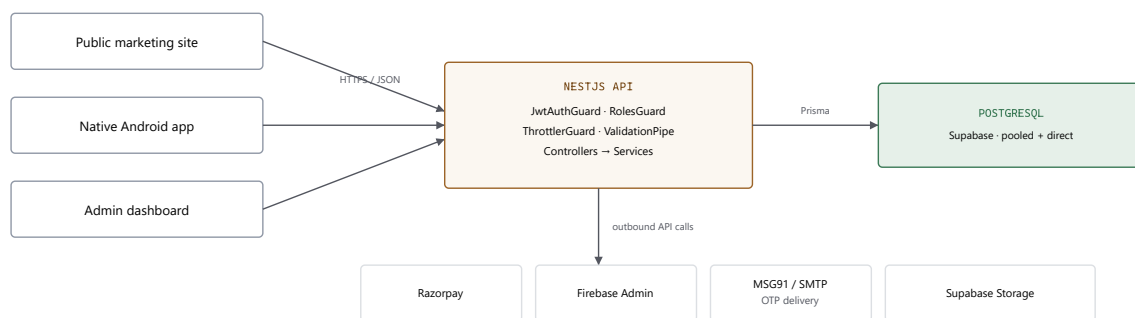


# ARCHITECTURE OVERVIEW

*How the pieces fit together: three client surfaces, one API, one database.*

## 1.1 – SYSTEM OVERVIEW & REQUEST FLOW

Kee is a single-tenant-per-shop SaaS platform for duplicate-key shops, built as a NestJS REST API backed by PostgreSQL, served to three client surfaces from one codebase and one backend: the public marketing/directory site, the installed Android app (Capacitor-wrapped React), and the authenticated Shop Admin / Super Admin dashboard. All three talk to the same API over HTTPS/JSON; there is no GraphQL layer and no server-rendered pages — the backend is a pure API server.



**Fig. 1** – Request flow from any of the three client surfaces through the API's guard/validation pipeline to PostgreSQL, with outbound calls to third-party services on the side.

Multi-tenancy is enforced at the service layer, not by separate databases: every Shop Admin-scoped query is filtered by the caller's `shopId` (read from the JWT, never from client input), and every read additionally excludes soft-deleted rows (`deletedAt IS NULL`) via a shared Prisma wrapper. Super Admin accounts have no `shopId` and can query across all shops.

## 1.2 – TECHNOLOGY STACK

### BACKEND

NestJS 10

Prisma 5

PostgreSQL (Supabase)

passport-jwt

@nestjs/throttler

class-validator

bcrypt

firebase-admin 14

razorpay SDK

nodemailer

@supabase/supabase-js

@nestjs/schedule

compression

TypeScript



## FRONTEND & MOBILE

React 18

Vite

Capacitor 8 (Android)

capacitor-razorpay

@capacitor-firebase/authentication

@capacitor/geolocation

@capacitor/camera

@capacitor/filesystem

@capacitor/share

jsPDF + html2canvas

lucide-react

Firebase JS SDK 12

The dashboard, the public marketing site, and the pre-login mobile shell all ship from one React codebase and one Vite build — there is no separate mobile framework. The native Android build wraps the same web bundle in a Capacitor shell and adds a small set of native plugins (camera, GPS, filesystem, share sheet, native Razorpay checkout, native Firebase phone auth) that the web build simply doesn't load.



## 1.3 – PROJECT STRUCTURE

### BACKEND – `BACKEND/SRC/`

|                                            |                                                                                                         |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <code>AUTH/</code>                         | Login, OTP send/verify, Firebase phone verification, shop self-registration, JWT strategy, guards       |
| <code>SHOP/</code>                         | Shop CRUD, settings, suspension, subscriptions, referrals, public shop directory & search               |
| <code>CUSTOMER/</code>                     | Compliance-record CRUD for Shop Admin and Super Admin, document/report uploads, public report downloads |
| <code>PROMOTION/</code>                    | Machines/Products/Offers/Ads listings (the <code>Promotion</code> model), public machines directory     |
| <code>AD/</code>                           | Super Admin advertisement campaigns (banner/popup/app-poster), public ad & poster endpoints             |
| <code>KEY/, KEY-TYPE/</code>               | Platform-wide Master Key catalogue and the curated key-type list                                        |
| <code>SHOP-CATEGORY/, PRODUCT-TYPE/</code> | Super Admin-curated taxonomy lists used across registration and filters                                 |
| <code>PAYMENT/</code>                      | Razorpay order creation and signature verification                                                      |
| <code>REPORT/</code>                       | Super/Shop dashboards, revenue log, support configuration                                               |
| <code>NOTIFICATION/</code>                 | In-app notification lists for Shop Admin and Super Admin                                                |
| <code>GEO/</code>                          | Reverse-geocoding proxy (Nominatim) backing every "Current Location" GPS autofill                       |
| <code>COMMON/, PRISMA/</code>              | Shared guards, decorators, the tenant-scoped Prisma wrapper, crypto & file-storage services             |

## FRONTEND – FRONTEND/SRC/

|                         |                                                                                                          |
|-------------------------|----------------------------------------------------------------------------------------------------------|
| APP.JSX                 | Authenticated shell: routing, the six-language dictionary, every Shop Admin / Super Admin dashboard view |
| PUBLICSITE.JSX          | Anonymous marketing/SEO site served at keyshops.in on the web build                                      |
| PUBLICMOBILEAPP.JSX     | Pre-login browse experience shown on the native app before sign-in                                       |
| COMPONENTS/             | Shared UI: registration wizards, OTP modal, image carousel/zoom, blog & location SEO pages               |
| CONTEXT/AUTHCONTEXT.JSX | Single API client — every network call in the app goes through this one module                           |
| UTILS/                  | PDF builders (customer report), SEO meta helpers, location datasets, shared platform/category helpers    |
| ANDROID/                | Generated Capacitor Android project — Gradle build, manifest, signing config                             |



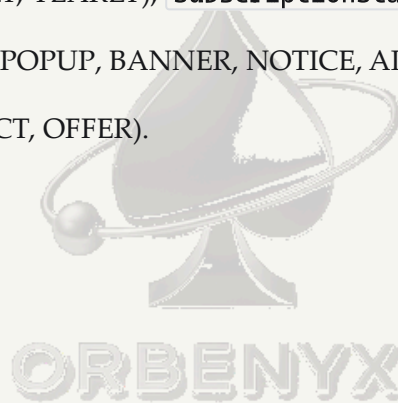


## DATA MODEL

24 Prisma models on PostgreSQL, one schema file, soft deletes throughout.

### 2.1 – DATABASE SCHEMA OVERVIEW

The schema is defined in a single `backend/prisma/schema.prisma` and applied via Prisma Migrate. Every tenant-owned table carries a nullable `deletedAt` column for soft deletes, and `createdAt` / `updatedAt` timestamps are automatic. Five enums constrain state: `Role` (SUPER\_ADMIN, SHOP\_ADMIN), `Plan` (TRIAL, MONTHLY, HALF\_YEARLY, YEARLY), `SubscriptionStatus` (ACTIVE, SUSPENDED, EXPIRED), `AdType` (POPUP, BANNER, NOTICE, APP\_POSTER), and `PromotionType` (AD, PRODUCT, OFFER).



| MODEL                                     | PURPOSE                                                                           |
|-------------------------------------------|-----------------------------------------------------------------------------------|
| <b>User</b>                               | Login account — email and/or phone, bcrypt password hash, one of two roles        |
| <b>Shop</b>                               | The tenant itself — profile, GPS/town/district, category, referral code & balance |
| <b>Referral</b>                           | Append-only ledger, one row per successfully referred shop                        |
| <b>ShopCategory, ProductType, KeyType</b> | Super Admin–curated taxonomy lists, orderable and soft-deletable                  |
| <b>Subscription</b>                       | A shop's paid period — plan, start/end date, status                               |
| <b>MasterKey</b>                          | Platform-wide, cross-shop catalogue of cut keys by category and key number        |
| <b>Customer</b>                           | The core compliance record — one row per key-duplication/loss visit               |
| <b>CustomerDocument, CustomerReport</b>   | Uploaded ID documents and the generated compliance PDF, per customer visit        |
| <b>ShopDocument</b>                       | Shop-level supporting documents (licence, ID proof)                               |
| <b>OtpCode</b>                            | Hashed, expiring OTP codes for phone/email verification                           |
| <b>Advertisement</b>                      | Super Admin campaigns — banner/popup/app-poster, date range, targeting            |
| <b>Promotion</b>                          | Unified model for Machines/Products, Offers, and shop-authored ad listings        |
| <b>ActivityLog</b>                        | Audit trail — action, PII-masked details, IP address                              |
| <b>Notification</b>                       | In-app notifications, shop-scoped or system-wide                                  |

| MODEL                 | PURPOSE                                                                                  |
|-----------------------|------------------------------------------------------------------------------------------|
| <b>RevenueRecord</b>  | Manually logged monthly platform revenue entries                                         |
| <b>PlatformConfig</b> | Single-row global settings — subscription price, GST %, support contact, WhatsApp number |

## 2.2 – CORE MODELS & RELATIONSHIPS

**Shop → User (1:N)** — a shop can have more than one login, though in practice each shop currently has exactly one Shop Admin user. **Shop → Customer / MasterKey / Promotion / ShopDocument / Notification (1:N)** — every one of these is scoped by `shopId` and filtered accordingly in every Shop Admin query.

**Customer → CustomerDocument / CustomerReport (1:N)** — a visit can carry multiple uploaded ID photos but a single generated compliance PDF. **Shop → ShopCategory (N:1, SetNull on delete)** — deleting a category does not cascade-delete shops; they simply become uncategorized. **Shop → Referral (1:N as referrer, 1:1 as referred)** — each shop can refer many others but was itself referred at most once, enforced by a unique constraint on `referredShopId`.

### NOTABLE FIELD DESIGN

`Customer.idProofNumber` and `Shop.aadhaarNumber` are encrypted at rest through a shared `CryptoService` (backed by the `ENCRYPTION_KEY` environment variable) rather than stored as plain text. `Customer.masterKeyId` links a visit to its catalogue entry only when the visit is an Add Key with a recognised vehicle category — a Lost Key visit or an unrecognised category leaves it null and creates no catalogue entry, by design (see §8.3).

## AUTHENTICATION & AUTHORIZATION

*Stateless JWTs, two roles, and a guard pipeline applied consistently across every protected route.*

### 3.1 – LOGIN & TOKEN ISSUANCE

A user authenticates with either an email address or a phone number plus a password. `AuthService` looks up the matching `User` row, verifies the password with `bcrypt`, and on success signs a JWT (via `@nestjs/jwt`) carrying the user's id, role, and `shopId` (null for Super Admin) as claims. The token is returned to the client and attached as a `Bearer` header on every subsequent request; the API itself holds no server-side session state.

### 3.2 – GUARDS, ROLES & RATE LIMITING

Two roles exist: `SUPER_ADMIN` (platform operator, full cross-shop access) and `SHOP_ADMIN` (scoped to one shop). Protected controllers apply `JwtAuthGuard` (validates the token's signature and expiry) and `RolesGuard` (reads a `@Roles(...)` decorator via Nest's `Reflector` and compares it against the token's role claim) — a route with no `@Roles` decorator is reachable by any authenticated role, and a route with no guards at all is public.

| ROUTE                                         | LIMIT       | WINDOW | RATIONALE                              |
|-----------------------------------------------|-------------|--------|----------------------------------------|
| <code>POST /auth/login</code>                 | 10 requests | 10 min | Slow credential-stuffing / brute-force |
| <code>POST /auth/send-otp</code>              | 6 requests  | 10 min | Limit SMS/email delivery cost and spam |
| <code>POST /auth/verify-otp</code>            | 10 requests | 10 min | Slow OTP-guessing attempts             |
| <code>POST /auth/reset-password-public</code> | 6 requests  | 10 min | Slow password-reset abuse              |

Every DTO-bound request body is validated by a global `ValidationPipe` configured with `whitelist: true`, `transform: true`, and `forbidNonWhitelisted: true` — unknown fields are rejected outright rather than silently dropped, and every input is checked against its `class-validator` decorators before a controller method ever runs.

### 3.3 – OTP DELIVERY & FIREBASE PHONE AUTH

Phone-based OTPs are delivered primarily through **MSG91** (India-specific SMS provider, DLT-registered template required); email OTPs fall back to SMTP via `nodemailer`. The OTP itself is always generated and bcrypt-hashed server-side and stored in `OtpCode` — the third-party provider is purely a delivery channel, never the source of truth for verification. If no provider is configured, delivery silently no-ops and the code is only ever visible in server logs, never echoed back to the client.

The native Android app additionally supports Firebase Phone Authentication as a parallel path: the app performs the phone verification client-side via the Firebase SDK, then sends the resulting ID token to `POST /auth/verify-firebase-phone`, where `firebase-admin` verifies it server-side before issuing Kee's own JWT. This

avoids a second MSG91 SMS on devices where Firebase's own SMS verification already ran.



## API REFERENCE

Grouped by domain. Role pills show the minimum required role; Public means no token is required.

### 4.1 – AUTHENTICATION ENDPOINTS

| METHOD | PATH                        | ACCESS            | PURPOSE                                      |
|--------|-----------------------------|-------------------|----------------------------------------------|
| POST   | /auth/login                 | Public            | Email/phone + password login                 |
| POST   | /auth/send-otp              | Public            | Send SMS/email OTP                           |
| POST   | /auth/verify-otp            | Public            | Verify OTP code                              |
| POST   | /auth/verify-firebase-phone | Public            | Verify a Firebase phone-auth ID token        |
| POST   | /auth/register-shop         | Public            | Self-registration, gated on verified payment |
| POST   | /auth/reset-password-public | Public            | Reset password after OTP verification        |
| POST   | /auth/change-password       | Any authenticated | Change password while logged in              |
| POST   | /auth/update-credentials    | Any authenticated | Update phone/email, OTP-verified             |
| GET    | /auth/me                    | Any authenticated | Current user profile                         |

## 4.2 – SHOP & SHOP CATEGORY ENDPOINTS

| METHOD              | PATH                           | ACCESS | PURPOSE                                    |
|---------------------|--------------------------------|--------|--------------------------------------------|
| GET                 | /super/shops                   | Super  | Paginated shop list, cursor-based          |
| POST                | /super/shops                   | Super  | Manually create a shop (support-case path) |
| GET                 | /super/shops/:id               | Super  | Shop detail                                |
| PUT                 | /super/shops/:id               | Super  | Edit shop                                  |
| POST                | /super/shops/:id/suspend       | Super  | Suspend / reactivate a shop                |
| POST                | /super/subscriptions/:shopId   | Super  | Create or renew a subscription             |
| GET / PUT           | /shop/settings                 | Shop   | View / edit own shop profile               |
| POST / DELETE       | /shop/settings/documents(/:id) | Shop   | Upload / remove a shop document            |
| POST                | /shop/settings/logo/upload     | Shop   | Upload shop logo                           |
| POST                | /shop/settings/referral        | Shop   | Apply a referral code                      |
| GET                 | /shop/referral                 | Shop   | Own referral code, balance, referred shops |
| GET                 | /shop-categories               | Public | List categories for registration/filters   |
| POST / PUT / DELETE | /super/shop-categories(/:id)   | Super  | CRUD + drag-and-drop reorder               |



## 4.3 – CUSTOMER & COMPLIANCE RECORD ENDPOINTS

| METHOD              | PATH                                   | ACCESS | PURPOSE                                            |
|---------------------|----------------------------------------|--------|----------------------------------------------------|
| POST                | /shop/customers                        | Shop   | Create a compliance record                         |
| GET                 | /shop/customers                        | Shop   | List / search own shop's records                   |
| GET                 | /shop/customers/global-search          | Shop   | Search across the shop's history                   |
| POST /<br>DELETE    | /shop/customers/:id/docs(/:docId)      | Shop   | Upload / remove an ID document                     |
| POST                | /shop/customers/:id/report             | Shop   | Store the generated compliance PDF                 |
| PUT                 | /shop/customers/:id                    | Shop   | Edit a record                                      |
| GET / POST /<br>PUT | /super/customers(/:id)                 | Super  | Platform-wide customer registry, cross-shop        |
| POST                | /super/customers/:id/docs ,<br>/report | Super  | Same uploads, Super Admin path                     |
| GET                 | /public/reports/:id/download           | Public | Download a stored compliance PDF via its opaque id |

## 4.4 – MACHINES, PROMOTIONS & KEYS ENDPOINTS

| METHOD                 | PATH                                       | ACCESS               | PURPOSE                                                  |
|------------------------|--------------------------------------------|----------------------|----------------------------------------------------------|
| GET                    | /promotions                                | Shop /<br>Super      | Role-scoped listing feed                                 |
| POST                   | /shop/promotions/upload-image              | Shop                 | Upload listing image (max 4 photos)                      |
| POST / PUT<br>/ DELETE | /shop/promotions(/:id)                     | Shop                 | Machine/product listing CRUD                             |
| same trio              | /super/promotions(/:id)                    | Super                | Super Admin–authored listings                            |
| GET                    | /public/machines(/:id)                     | Public               | Public Machines/Products directory, town/district filter |
| GET                    | /product-types                             | Public               | Curated product-type list                                |
| POST / PUT<br>/ DELETE | /super/product-types(/:id)                 | Super                | Product-type CRUD                                        |
| GET                    | /key-types                                 | Any<br>authenticated | Curated key-type list                                    |
| POST / PUT<br>/ DELETE | /super/key-types(/:id)                     | Super                | Key-type CRUD                                            |
| GET                    | /super/keys ,<br>/super/shops/:shopId/keys | Super                | Platform-wide Master Key catalogue                       |
| GET                    | /shop/keys/search                          | Shop                 | Search own shop's cut keys                               |

## 4.5 – ADVERTISEMENTS & NOTIFICATIONS

| METHOD              | PATH                               | ACCESS | PURPOSE                                  |
|---------------------|------------------------------------|--------|------------------------------------------|
| GET                 | /super/advertisements              | Super  | List campaigns                           |
| POST                | /super/advertisements/upload-image | Super  | Upload campaign creative                 |
| POST / PUT / DELETE | /super/advertisements(/:id)        | Super  | Campaign CRUD                            |
| GET                 | /shop/advertisements               | Shop   | Ads targeted at own shop                 |
| GET                 | /public/ads                        | Public | Active ads for the public app carousel   |
| GET                 | /public/ads/poster                 | Public | Full-screen app-open poster (APP_POSTER) |
| GET / PUT           | /shop/notifications(/:id)          | Shop   | List / mark-read own notifications       |
| GET / PUT           | /super/notifications(/:id)         | Super  | List / mark-read platform notifications  |

### 4.6 – PAYMENTS & PLATFORM CONFIGURATION

| METHOD        | PATH                                  | ACCESS           | PURPOSE                                               |
|---------------|---------------------------------------|------------------|-------------------------------------------------------|
| POST          | /payment/create-order                 | Public           | Create a Razorpay order for a new shop's subscription |
| GET           | /super/dashboard ,<br>/shop/dashboard | Role-<br>matched | Summary metrics for each dashboard                    |
| GET /<br>POST | /super/revenue                        | Super            | View / log monthly revenue entries                    |
| POST          | /super/support-config                 | Super            | Update support contact / WhatsApp / videos            |

### 4.7 – PUBLIC (UNAUTHENTICATED) ENDPOINTS

These routes carry no JWT requirement by design — they power the anonymous marketing site, the pre-login native app shell, and the public key-shop directory.

|                         |                                  |                      |                        |
|-------------------------|----------------------------------|----------------------|------------------------|
| GET /public/shops(/:id) | GET /public/machines(/:id)       | GET /public/ads      | GET /public/ads/poster |
| GET /public/search      | GET /public/reports/:id/download | GET /shop-categories | GET /product-types     |
| GET /support-config     | GET /geo/reverse-geocode         | GET /health          |                        |

## INTEGRATIONS & CONFIGURATION

*Five external services, all optional at boot and safely no-op when unconfigured.*

### 5.1 – THIRD-PARTY INTEGRATIONS

#### RAZORPAY

##### PAYMENTS

Powers self-service shop registration. The backend creates an order ( `POST /payment/create-order` ), the client completes checkout via the Razorpay SDK (web) or the `capacitor-razorpay` native plugin, and the resulting payment id/order id/signature are submitted with the registration form. `AuthService` re-verifies the signature server-side with HMAC-SHA256 before any account row is created — the client-reported "success" is never trusted on its own.

#### FIREBASE

##### NATIVE PHONE AUTH & PUSH-READY INFRA

`firebase-admin` verifies ID tokens produced by the native app's client-side Firebase phone verification (an alternative OTP path to MSG91). Configured via a service-account JSON pasted directly into an environment variable, since the hosting platform has no persistent disk.

#### MSG91 / SMTP

##### OTP DELIVERY

MSG91 delivers SMS OTPs for phone-based login, registration, and password reset (DLT-registered template required for Indian numbers); SMTP via `nodemailer` is the parallel channel for email-based flows. The OTP code itself is always generated and verified by the backend — both providers are delivery-only.

## SUPABASE

### POSTGRES HOSTING & OBJECT STORAGE

Supabase provides the managed PostgreSQL instance (pooled connection for runtime queries, direct connection for migrations) and Supabase Storage for uploaded files — shop logos, listing photos, customer ID documents/photos, and generated compliance-report PDFs.

## 5.2 – ENVIRONMENT VARIABLES

### SECURITY NOTE

The table below lists variable *names and purposes only*. No real secret values are reproduced in this document; every deployment must hold its own values in a secrets manager or platform environment configuration, never committed to source control.



| VARIABLE                                                                                                     | PURPOSE                                                                      |
|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <code>DATABASE_URL</code>                                                                                    | Pooled Postgres connection (Supervisor transaction mode) for runtime queries |
| <code>DIRECT_URL</code>                                                                                      | Direct Postgres connection, required for <code>prisma migrate</code>         |
| <code>JWT_SECRET</code>                                                                                      | Signing key for issued JWTs                                                  |
| <code>ENCRYPTION_KEY</code>                                                                                  | Key for at-rest encryption of ID-proof numbers and Aadhaar numbers           |
| <code>PORT</code> , <code>NODE_ENV</code>                                                                    | Server port and environment mode                                             |
| <code>SUPABASE_URL</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code> ,<br><code>SUPABASE_STORAGE_BUCKET</code> | Object storage connection for uploaded files                                 |
| <code>SEED_SUPER_ADMIN_EMAIL/PASSWORD/NAME</code>                                                            | Bootstrap credentials for the first Super Admin account                      |
| <code>SMTP_HOST/PORT/SECURE/USER/PASS/FROM</code>                                                            | Email OTP and notification delivery                                          |
| <code>MSG91_AUTH_KEY</code> , <code>MSG91_OTP_TEMPLATE_ID</code>                                             | SMS OTP delivery                                                             |
| <code>RAZORPAY_KEY_ID</code> , <code>RAZORPAY_KEY_SECRET</code>                                              | Payment order creation and signature verification                            |
| <code>FIREBASE_SERVICE_ACCOUNT_JSON</code>                                                                   | Firebase Admin SDK credential (raw JSON, not a file path)                    |

## DEPLOYMENT & OPERATIONS

*One backend service, one static frontend host, one Android build pipeline.*

### 6.1 – DEPLOYMENT TOPOLOGY

|              |                                                                                                                                                                |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BACKEND API  | NestJS server on a managed Node hosting platform (no persistent local disk — all file storage goes to Supabase Storage, all secrets are environment variables) |
| DATABASE     | Supabase-managed PostgreSQL, pooled connection for the app and a direct connection reserved for migrations                                                     |
| WEB FRONTEND | Static React/Vite build deployed to Firebase Hosting, serving both the public marketing site and the authenticated dashboard from one bundle                   |
| NATIVE APP   | Signed Android release APK, built via Capacitor + Gradle, distributed outside the Play Store via a direct download link on the marketing site                  |

### 6.2 – BUILD & RELEASE PIPELINE

- 1 Frontend build.** `npm run build` from `frontend/` compiles the Vite bundle; the project-root `index.html` is the real build entry (not anything under `public/`).
- 2 Web deploy.** `firebase deploy --only hosting` publishes the built `dist/` folder.
- 3 Native sync.** `npx cap sync android` copies the fresh web bundle into the Capacitor Android project and updates native dependencies.
- 4 Android build.** Gradle assembles and signs a release APK ( `gradlew assembleRelease` ) using the project's release keystore.
- 5 Distribution.** The signed APK is uploaded to Firebase Hosting alongside the web build, and the marketing site's download link is verified to point at the new file.



6

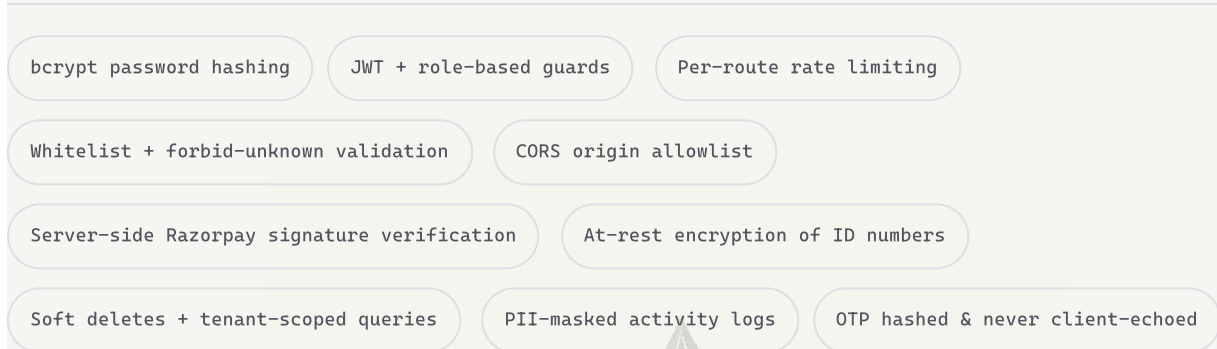
**Backend deploy.** The API redeploys from its connected Git branch; any new Prisma migration is applied via `prisma migrate deploy` before the new code starts serving traffic.



## SECURITY CONSIDERATIONS

*What's already enforced, and what a client audit should flag for follow-up.*

### 7.1 – IMPLEMENTED CONTROLS



CORS is enforced with an explicit origin-allowlist callback rather than a wildcard,

so only the known web and native origins can call the API with credentials.

Request bodies pass through a global `ValidationPipe` before reaching any handler, and every tenant-scoped read is filtered server-side by the caller's JWT-derived `shopId` — a Shop Admin cannot widen a query by manipulating request parameters.

### 7.2 – KNOWN LIMITATIONS & HARDENING NOTES

#### JWT SECRET FALLBACK

The JWT strategy falls back to a hardcoded default secret if `JWT_SECRET` is not set in the environment. This is safe only because every real deployment sets the variable explicitly — it should be changed to a hard failure on boot (refuse to start without a configured secret) as a follow-up hardening item.

#### RECOMMENDED NEXT STEPS

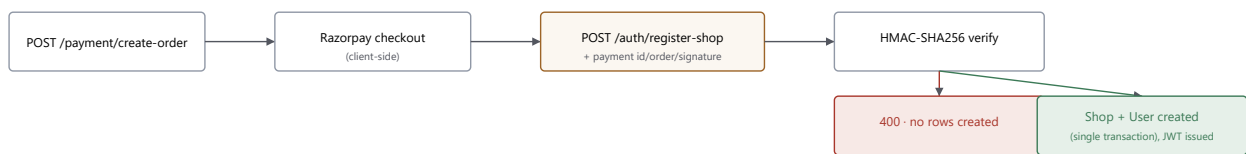
For a client handover, we recommend: (1) rotating `JWT_SECRET` and `ENCRYPTION_KEY` before go-live and storing them in a secrets manager; (2) enabling Prisma query logging/alerting in production for anomaly detection; (3) adding automated dependency vulnerability scanning to the release pipeline; (4) a periodic access review of Super Admin accounts, since that role has unrestricted cross-shop access.



## TECHNICAL WORKFLOWS

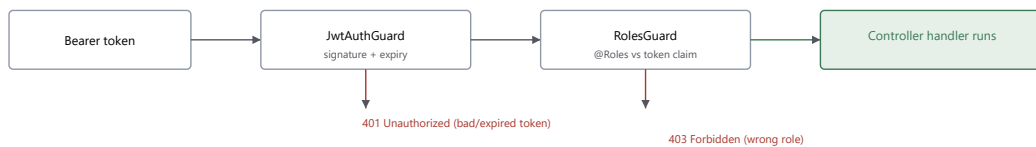
Three flows that touch the most moving parts in the system.

### 8.1 – SHOP REGISTRATION & PAYMENT SEQUENCE



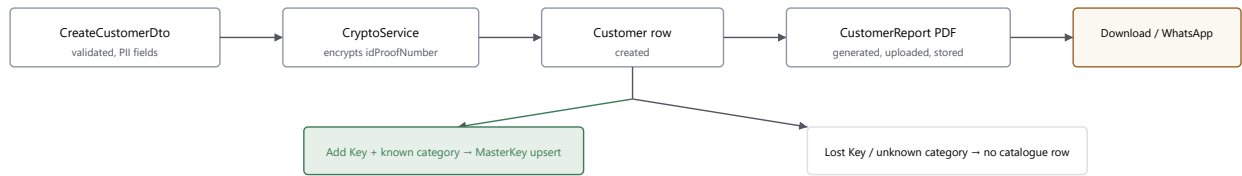
**Fig. 2** – The backend never trusts a client-reported payment success; it recomputes the Razorpay signature itself before any account row exists.

### 8.2 – REQUEST AUTHENTICATION FLOW



**Fig. 3** – Every protected route runs the same two-guard pipeline before its handler is reached.

### 8.3 – CUSTOMER VISIT & COMPLIANCE RECORD FLOW



**Fig. 4** – The Master Key catalogue auto-create is gated behind both the Add Key flag and a recognised vehicle category, so Lost Key visits and unrecognised categories never pollute the cross-shop catalogue.

#### PLACEHOLDER COMPANY DETAILS

Every business detail on the cover page — address, phone, email, website — is a placeholder for Orbenyx's real information. Replace before sending to the client.

Environment-variable values throughout this document are names and purposes only; no real secrets are reproduced here.

